

COMPLETE EXAM STUDY GUIDE

CMS707

Artificial Intelligence

 **Course:** CMS707 — Artificial Intelligence

 **Lecturer:** Dr. Friday Orji

 **Institution:** University of Science & Technology

 **Session:** 2025 / 2026

Cheat Sheet Prepared by
Mbosinwa Awunor

Search Algorithms

Knowledge Rep.

Predicate Logic

AI Agents

Machine Learning

What is Artificial Intelligence?

Intelligence — The capacity to think & comprehend the world in an elaborate way, involving *knowledge, experience and understanding*.

Artificial Intelligence — Building systems that act *rationally* to maximally achieve predefined goals; understanding human cognition and replicating it — but *FASTER*.

Turing Test — Can a machine converse indistinguishably from a human? The benchmark for machine intelligence.

MUNDANE TASKS (HARD FOR COMPUTERS)

- Perception & Vision
- Speech & Natural Language
- Common Sense Reasoning
- Translation, Generation
- Robot Control

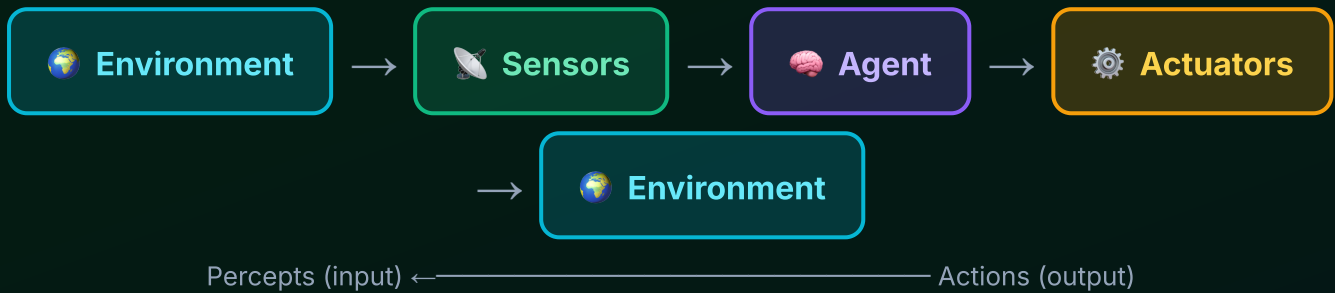
FORMAL TASKS (EASY FOR COMPUTERS)

- Chess, Backgammon, Checkers
- Mathematics & Calculus
- Logic & Theorem Proof
- Computation & Arithmetic

Key Insight: Mundane tasks require much more knowledge to represent in a computer than formal tasks. Humans do mundane things effortlessly — machines struggle.

Agents & Rational Agents

Agent — Something that *perceives* its environment via **sensors** and *acts* upon it via **actuators**. Operates automatically, persists over time, adapts.



TERM	DEFINITION
Percept Sequence	Complete history of <i>everything</i> the agent has ever perceived
Agent Function	Abstract mathematical mapping: percept sequence $\rightarrow$ action
Agent Program	Concrete code implementation of the agent function
Rational Agent	Acts to achieve the <i>best/expected outcome</i> , judged by a performance measure
Performance Measure	Criterion capturing what "best outcome" means for a rational agent

Don't confuse: Agent Function = abstract maths. | Agent Program = actual runnable code. Rational Agent $\neq$ "smart" — it means *goal-maximising under its performance measure*.

State Space Search — 4 Steps

State Space — Set of all possible states in a problem, connected by operators. Any problem can be defined as *initial state* → *goal state*.

- 1 Define the Problem Precisely** — Specify the exact initial situation(s) and what constitutes an acceptable solution (goal state).

Example: FLGY → ?? → |FLGY

- 2 Analyse the Problem** — A few key features have immense impact on which technique to use.
- 3 Isolate & Represent** the task knowledge necessary to solve the problem.
- 4 Choose & Apply** the best problem-solving technique for the problem.

Search is critical for hard problems where no direct technique is available. The state space represents *all reachable states* from the initial state.

Water Jug & Farmer's Problem



WATER JUG PROBLEM

4-gallon jug + 3-gallon jug (no markings).
Get exactly **2 gallons** in the 4-gal jug.

STATE SPACE

State = **(x, y)** where
 $x \in \{0,1,2,3,4\} \mid y \in \{0,1,2,3\}$

START & GOAL

Start: (0, 0)

Goal: (2, 3) ✓

Solution path:

(0,0) → (0,3) → (3,0) → (3,3)
 → (4,2) → (0,2) → (2,0) → **(2,3) ✓**



FARMER'S PROBLEM

Farmer + Lion + Goat + Yam.
Boat holds farmer + **one item** only.

Constraints — Never leave alone:

🦁 Lion + Goat (Lion eats Goat)

🐐 Goat + Yam (Goat eats Yam)

STATE NOTATION

FLGY| — all on left bank

|FLGY — all on right bank

Bar | represents the river

Key move: Take the Goat first, then go back to get Lion or Yam, bring Goat back, swap — this is the trick!

BFS vs DFS — The Core Trade-off

BREADTH-FIRST SEARCH (BFS)

- Uses a **FIFO Queue**
- Explores *level by level*
-  Complete (always finds solution)
-  Optimal (shortest path)
-  High memory usage
- Must store entire explored tree

Best when: Solution is shallow, memory is not a constraint.

DEPTH-FIRST SEARCH (DFS)

- Uses a **LIFO Stack**
- Explores *one branch fully*
-  Not always complete (can loop)
-  Not optimal
-  Low memory (current path only)
-  Fast when many solutions exist

Best when: Memory is limited, many acceptable solutions exist.

BEST OF BOTH WORLDS

★ ITERATIVE DEEPENING (IDS)

DFS with increasing depth limit.

 Complete  Optimal  Low memory
BFS completeness + DFS memory efficiency

BEST-FIRST / A* SEARCH

Uses a **heuristic** to guide search.

Combines BFS safety + DFS efficiency.

A*: $f(n) = g(n) + h(n)$

06 — SEARCH ALGORITHMS

Full Algorithm Comparison

ALGORITHM	STRUCTURE	COMPLETE?	OPTIMAL?	MEMORY	TYPE
BFS	FIFO Queue	✓ Yes	✓ Yes	High	Uninformed
DFS	LIFO Stack	✗ No	✗ No	Low	Uninformed
Iterative Deepening	Stack + depth	✓ Yes	✓ Yes	Low	Uninformed
Best-First	Priority Queue	✓ Yes	✓ Yes	Medium	Informed
A*	Priority Queue	✓ Yes	✓ Yes*	Medium	Informed

$$f(n) = g(n) + h(n)$$

$g(n)$ = actual cost from start to node n | $h(n)$ = estimated cost from n to goal (heuristic)
* A* is optimal when heuristic $h(n)$ is **admissible** (never overestimates)

Heuristic — Problem-specific knowledge that estimates the cost/distance to the goal. Informed search uses heuristics; uninformed search (BFS, DFS) does not.

BFS & DFS Pseudocode

BFS — FIFO QUEUE

```

BREADTH-FIRST-SEARCH(Problem):
  node ← NODE(Problem.INITIAL)
  if IS-GOAL(node): return node
  frontier ← FIFO queue // Queue!
  reached ← {Problem.INITIAL}

  while not EMPTY(frontier):
    node ← POP(frontier)
    for child in EXPAND(node):
      s ← child.STATE
      if IS-GOAL(s): return child
      if s ∉ reached:
        ADD s to reached
        ADD child to frontier

  return failure

```

DFS — ITERATIVE DEEPENING

```

ITERATIVE-DEEPENING(Problem):
  for depth = 0 to ∞:
    result ← DEPTH-LIMITED(Problem, depth)
    if result ≠ cutoff:
      return result

DEPTH-LIMITED(Problem, L):
  frontier ← LIFO stack // Stack!
  result ← failure

  while not EMPTY(frontier):
    node ← POP(frontier)
    if IS-GOAL(node): return node
    if DEPTH(node) ≥ L:
      result ← cutoff
    else if not IS-CYCLE(node):
      ADD children to frontier

  return result

```

Memory trick: BFS = Broad → Queue (FIFO). DFS = Deep → Stack (LIFO). Iterative Deepening = DFS controlled by a depth limit that increases each round.

Knowledge Representation (KR)

Fact — Truth in the real world
(what we want to represent)
Representation — The
formalism we actually
manipulate

Hallucination — Representing facts that
don't actually exist. Machines have no
consciousness or self-awareness to
ground their representations in reality.

**FLOW: NATURAL LANGUAGE →
INTERNAL**

English: "Socrates is a man"

↓ Internal: `Man(Socrates)`

↓ Logic: `∀x: man(x) → mortal(x)`

↓ Inferred: `mortal(Socrates)`

↓ English: "Socrates is mortal"

FORM

INFERENCE POWER

**Natural
Language**

Weak — Ambiguous,
hard to process

**Simple
Relational (DB)**

Limited — Tables, basic
queries

**Inferential /
Inheritance**

Strong — Class
hierarchies, inherit
properties

**Predicate
Logic**

Strongest — Formal,
complete, used in AI
inference

SIMPLE RELATIONAL EXAMPLE (DB)

Player table with name, height, weight,
position.

Can answer: "Who is the heaviest player?"

Cannot: infer class membership or reason
logically.

INFERENCE / INHERITANCE

Person → Athletic Male → Football Player
→ Midfielder (Botton, Bkacha)
→ Attacker (Messi, Etisn)
Properties inherited down the hierarchy.

Predicate Logic — Syntax & Symbols

```
Sentence → Atomic | Complex

Atomic → Predicate(Term) | Term = Term

Complex →
  ¬ Sentence
  | Sentence ∧ Sentence
  | Sentence ∨ Sentence
  | Sentence ⇒ Sentence
  | Sentence ⇔ Sentence
  | ∀ Variable, Sentence
  | ∃ Variable, Sentence

Term → Constant | Variable | Fn(Term)
```

Constants — A, X, John, Marcus, Caesar
Variables — x, y, a, s (lower case)
Predicates — Man(), Ruler(), loyal(), hate()
Functions — Mother(x), Little(x)

EXAMPLES FROM LECTURE

```
Man(Marcus)
∀x Pompeian(x) → Roman(x)
∀x ∃y loyal_to(x, y)
∀x: man(x) → mortal(x)
mortal(Socrates)
```

Why Predicate Logic?
It bridges natural language and machine reasoning. It supports *formal inference* — we can derive new facts from known facts automatically.

SYMBOL	MEANING
∀	For all (universal)
∃	There exists (existential)
∧	AND (conjunction)
∨	OR (disjunction)
¬	NOT (negation)
⇒	Implies (if → then)
⇔	If and only if

Marcus & Caesar — Worked Example

8 AXIOMS (GIVEN FACTS)

1. $\text{Man}(\text{Marcus})$
2. $\text{Pompeian}(\text{Marcus})$
3. $\forall x \text{ Pompeian}(x) \rightarrow \text{Roman}(x)$
4. $\text{Ruler}(\text{Caesar})$
5. $\forall x \text{ Roman}(x) \rightarrow [\text{loyal}(x, \text{Caesar}) \vee \text{hate}(x, \text{Caesar})]$
6. $\forall x \exists y \text{ loyal_to}(x, y)$
7. $\forall x \forall y \text{ person}(x) \wedge \text{ruler}(y) \wedge \text{try_assassinate}(x, y) \rightarrow \neg \text{loyal_to}(x, y)$
8. $\text{try_assassinate}(\text{Marcus}, \text{Caesar})$

INFERENCE CHAIN



Axiom 2+3: Marcus is Pompeian $\rightarrow$
Marcus is Roman



Axiom 5: Roman $\rightarrow$ **loyal OR hates Caesar**



Axiom 8: Marcus tried to assassinate Caesar



Axiom 7: Assassinated ruler $\rightarrow$ **NOT loyal**



Since not loyal $\rightarrow$ from Axiom 5:
Marcus HATED Caesar

$\therefore \text{hate}(\text{Marcus}, \text{Caesar}) \checkmark$

Part III Overview — Advanced AI

MACHINE LEARNING

- **Supervised** — labelled data
- **Unsupervised** — patterns
- **Reinforcement** — reward/penalty
- Decision Trees
- Neural Networks
- Deep Learning

NLP

- Speech Recognition
- Machine Translation
- Question Answering
- *e.g. IBM Watson*
- Language Models
- Text Generation

EXPERT SYSTEMS

- Knowledge Base
- Inference Engine
- Forward Chaining
- Backward Chaining
- Rule-Based Systems
- Knowledge-Based AI

FUZZY LOGIC

- Truth values 0 to 1
- Not just True/False
- Handles *vagueness*
- Fuzzy Sets
- Fuzzy Inference
- vs Classical Boolean

GENETIC ALGORITHMS

- **Selection** — best survive
- **Crossover** — combine genes
- **Mutation** — random change
- Evolutionary optimisation
- Good for search/scheduling

PROBABILISTIC / BAYESIAN

- Handles uncertainty
- Prior knowledge + evidence
- Bayesian Networks
- Conditional probabilities
- Requires Predicate Logic

Semantic Nets — Graph-based KR: nodes = concepts, edges = relationships. Includes **Frames** and **Scripts**. Related to inheritance-based KR.

Common Exam Traps — Don't Confuse These

OFTEN CONFUSED	KEY DISTINCTION
BFS vs Best-First Search	BFS = uninformed breadth (queue). Best-First = heuristic-guided (priority queue). Different algorithms!
Agent Function vs Agent Program	Function = abstract mathematical description. Program = the actual runnable code.
Percept vs Percept Sequence	Percept = one observation at one moment. Sequence = the <i>full history</i> of all observations ever.
Rational ≠ Smart	Rational Agent = <i>goal-maximising under its performance measure</i> . Not necessarily intelligent or smart.
Inferential KR vs Predicate Logic	Predicate Logic is <i>one form</i> of inferential KR. Inferential KR is the broader category.
Iterative Deepening vs Repeated BFS	IDS runs DFS with increasing depth limits. It is NOT repeated BFS — it uses a stack, not a queue.
Supervised vs Reinforcement Learning	Supervised = learns from labelled data. Reinforcement = learns from rewards/penalties (no labels).
Fuzzy Logic vs Boolean Logic	Boolean = True/False only. Fuzzy = any value between 0 and 1. Handles vagueness and degrees.

Exam strategy: In predicate logic questions, write every step. Show the inference chain. In search questions, state the data structure (queue vs stack) and whether the algorithm is informed or uninformed.

Quick-Fire Definitions Glossary

TERM	ONE-LINE DEFINITION
Intelligence	Capacity to think & comprehend — via knowledge, experience, understanding
Agent	Perceives via sensors, acts via actuators, adapts & persists over time
Rational Agent	Acts to achieve best/expected outcome under a performance measure
Percept Sequence	Complete history of everything the agent has ever perceived
State Space	Set of all reachable states, connected by operators/actions
BFS	FIFO queue, level-by-level, complete & optimal, high memory
DFS	LIFO stack, depth-first, memory-efficient, may get trapped
Heuristic	Problem-specific estimate of cost/distance to goal
A*	$f(n) = g(n) + h(n)$; optimal when heuristic is admissible
KR	Mapping real-world facts into a formalism that can be manipulated
Predicate Logic	Formal system with $\forall/\exists$ quantifiers, predicates, constants, variables
Hallucination	Representing facts that don't exist; no self-aware grounding in machines
Fuzzy Logic	Truth values between 0 and 1; handles degrees of truth and vagueness
Genetic Algorithm	Evolutionary optimisation via selection, crossover, mutation
Turing Test	Can a machine converse indistinguishably from a human?

Recommended Study Order

1 What is AI?

Intelligence, Turing Test, Mundane vs Formal

2 Agents

Agent, Rational Agent, Percept Sequence, Performance Measure

3 State Space

4-step method, Water Jug, Farmer problem

4 BFS & DFS

Algorithms, trade-offs, pseudocode

5 Iterative Deepening + A*

Informed vs uninformed, heuristics

6 Knowledge Representation

Natural Lang → Relational → Inferential

7 Predicate Logic

Syntax + Marcus/Caesar worked example

8 Probabilistic Reasoning

Bayesian Networks, uncertainty

9 Semantic Nets

Frames, Scripts, graph-based KR

10 ML + Applications

NLP, Expert Systems, Fuzzy, Genetic Algorithms

PREPARED BY

Mbosinwa Awunor

CMS707 — Artificial Intelligence | Dr. Friday Orji | University of Science & Technology